A Report on ALM2 Project

A report submitted in partial fulfillment of the requirements
for the award of the degree of computer science and information technology

by

Student Name : A.Sahasra ID.NO : 2420090091

Dr.Jagan Mohan Reddy D



KONERU LAKSHMAIAH EDUCATION FOUNDATION, BOWRAMPET

Hyderabad-500043, Telangana, India

2026

Abstract

Project 1 : CryptoVault is an interactive web application developed to demonstrate the practical implementation of public-key cryptography and digital signature techniques. It allows users to register and log in, generate RSA key pairs, encrypt and decrypt messages, create and verify digital signatures, perform Diffie–Hellman key exchange, and generate SHA-512 message hashes.The application also supports public-key storage and self-signed X.509 certificate generation to demonstrate basic Public Key Infrastructure (PKI).

Project 2 : SecureTunnel is an educational web application designed to visualize and explain how secure communication channels are established using TLS/SSL, SSH, and IPsec concepts. It simulates the TLS handshake between a client and a server, demonstrating cipher-suite negotiation, server certificate verification, key establishment, session-key derivation, and handshake confirmation. It also illustrates the SSH connection process, including protocol negotiation, Diffie–Hellman key exchange, server authentication, and user authentication.

Contents

List of Tables **3**

List of Tables **4**

List of Tables

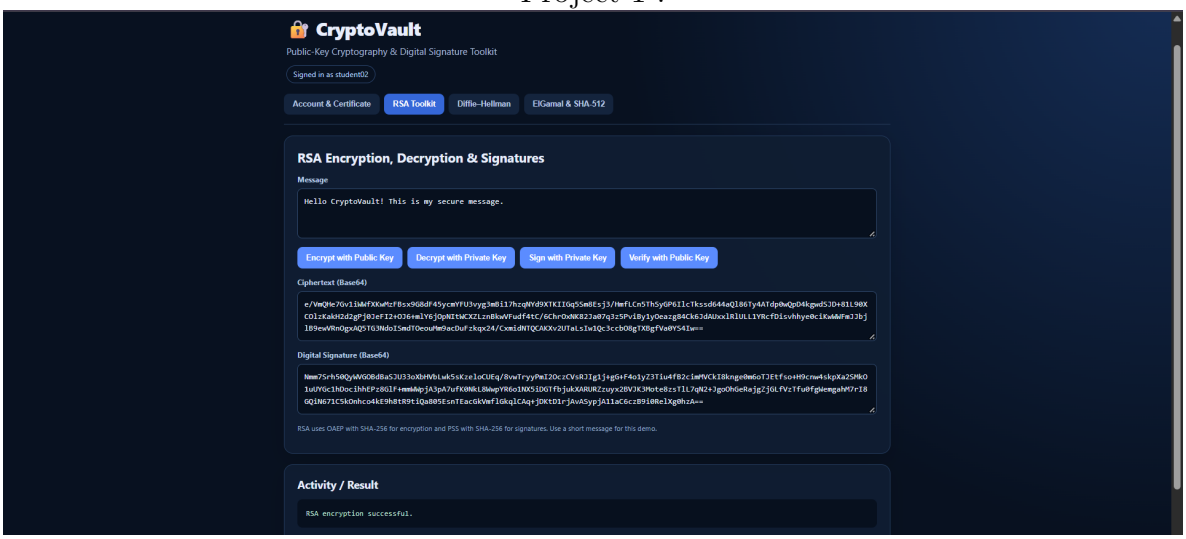
Project 1 : Component	Technology / Algorithm
Frontend UI	React
API Communication	Axios
Backend Framework	Python with Flask or FastAPI
Cryptographic Library	Python `cryptography`
Database	SQLite or PostgreSQL
User Authentication	Registration and login module
RSA Module	RSA algorithm
Key Exchange Module	Diffie–Hellman
Digital Signature Module	ElGamal signature scheme
Hashing Module	SHA-512 using `hashlib`
Certificate Module	X.509

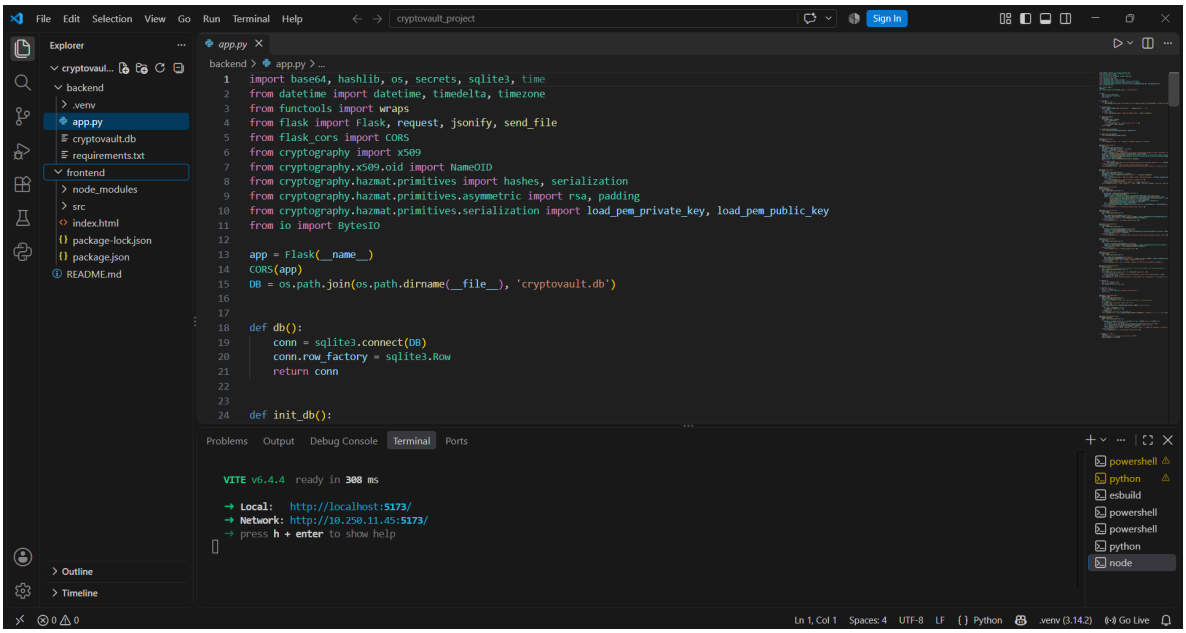
List of Tables

Project 2 : Component	Technology
Frontend UI	React
API Communication	Axios, if used
Backend Framework	Python with Flask or FastAPI
Cryptographic Library	Python `cryptography`
TLS Handshake Visualizer	Client–server interface
Certificate Verification Module	X.509 certificates
Key Exchange Module	RSA-based exchange or Diffie–Hellman
Session-Key Module	Key-derivation function
SSH Visualizer	Protocol and authentication steps
IPsec Visualizer	React Flow or Mermaid.js, if used
ESP Module	Encapsulating Security Payload
Handshake State Manager	Backend session/state logic
Step-by-Step Log	React components

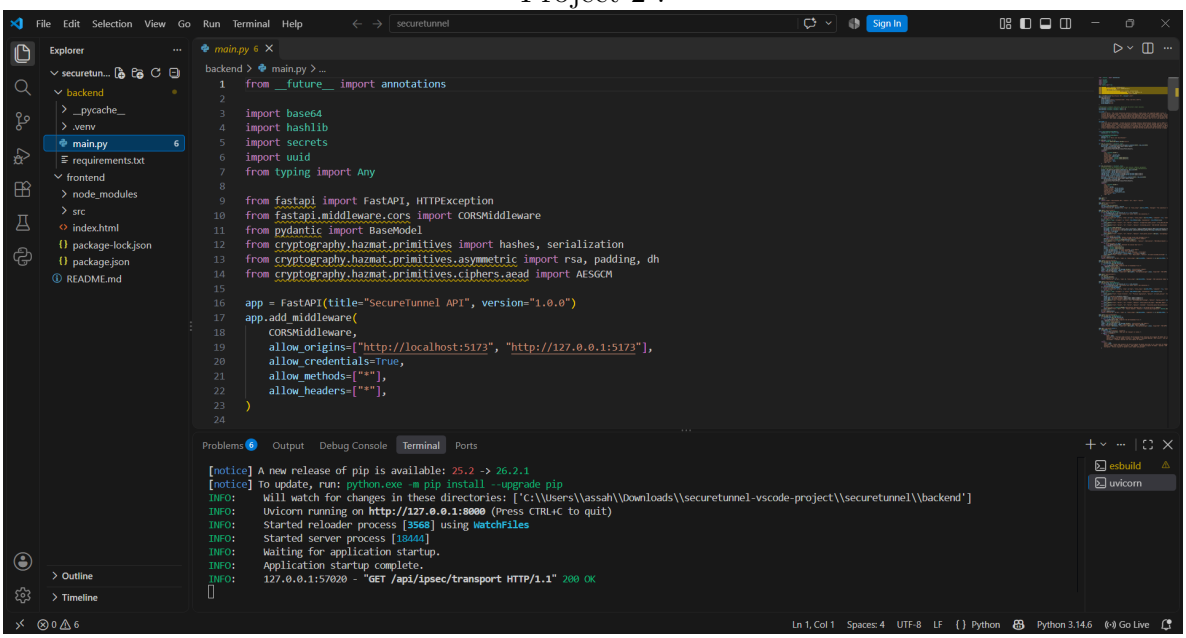
Screenshots

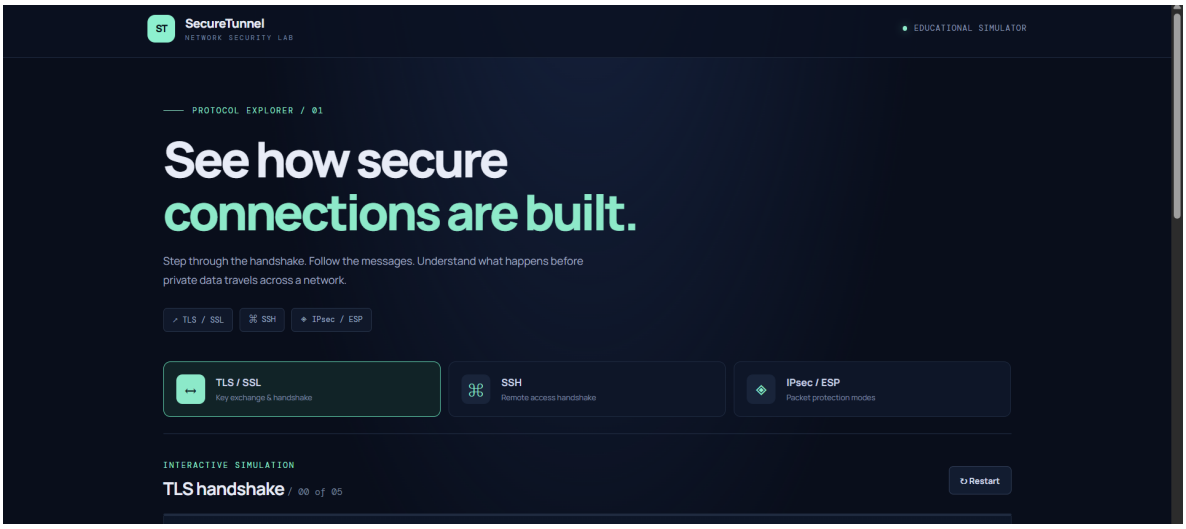
Project 1 :

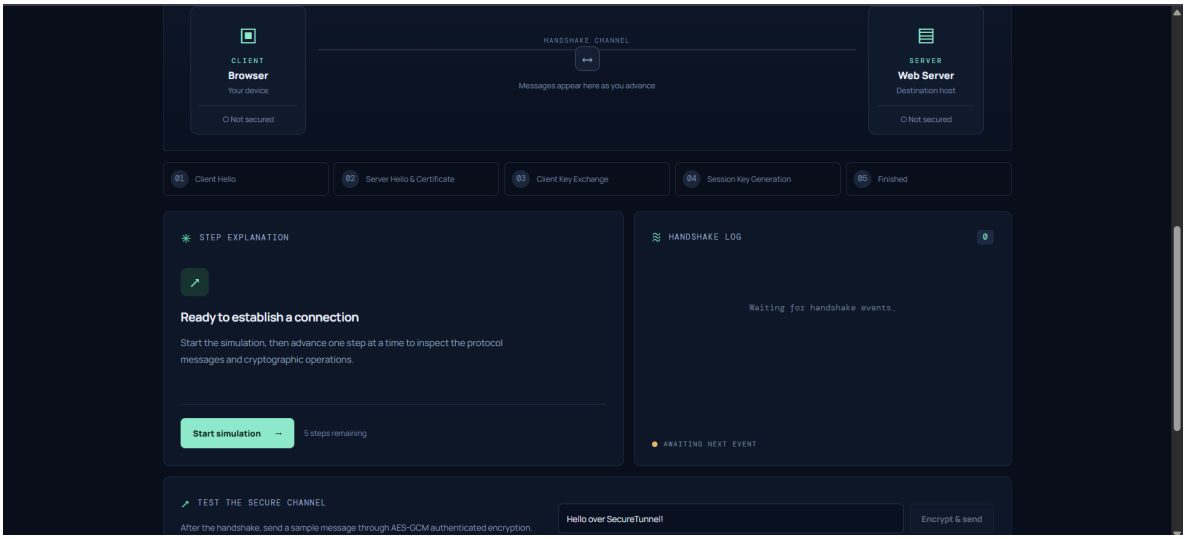



File Edit Selection View Go Run Terminal Help
cryptovault_project
Sign In
Explorer
app.py
backend > app.py > ...
import base64, hashlib, os, secrets, sqlite3, time
from datetime import datetime, timedelta, timezone
from functools import wraps
from flask import Flask, request, jsonify, send_file
from flask_cors import CORS
from cryptography import x509
from cryptography.x509.oid import NameOID
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives.serialization import load_pem_private_key, load_pem_public_key
from io import BytesIO
app = Flask(__name__)
CORS(app)
DB = os.path.join(os.path.dirname(__file__), 'cryptovault.db')
def db():
conn = sqlite3.connect(DB)
conn.row_factory = sqlite3.Row
return conn
def init_db():
backend
.venv
app.py
cryptovault.db
requirements.txt
frontend
node_modules
src
index.html
package-lock.json
package.json
README.md
Problems Output Debug Console Terminal Ports
VITE v6.4.4 ready in 308 ms
Local: http://localhost:5173/
Network: http://10.250.11.45:5173/
press h + enter to show help
powershell
python
esbuild
powershell
powershell
python
node
Outline
Timeline
Ln 1, Col 1 Spaces: 4 UTF-8 LF Python .venv (3.14.2) Go Live

Project 2 :


File Edit Selection View Go Run Terminal Help
securetunnel
Sign In
Explorer
main.py 6
backend > main.py > ...
from __future__ import annotations
import base64
import hashlib
import secrets
import uuid
from typing import Any
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import rsa, padding, dh
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
app = FastAPI(title="SecureTunnel API", version="1.0.0")
app.add_middleware(
CORSMiddleware,
allow_origins=["http://localhost:5173", "http://127.0.0.1:5173"],
allow_credentials=True,
allow_methods=["*"],
allow_headers=["*"],
)
backend
__pycache__
.venv
main.py 6
requirements.txt
frontend
node_modules
src
index.html
package-lock.json
package.json
README.md
Problems 6 Output Debug Console Terminal Ports
[notice] A new release of pip is available: 25.2 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
INFO: Will watch for changes in these directories: ['C:\\Users\\assah\\Downloads\\securetunnel-vscode-project\\securetunnel\\backend']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [3568] using WatchFiles
INFO: Started server process [18444]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:57020 - "GET /api/ipsec/transport HTTP/1.1" 200 OK
esbuild
uvicorn
Outline
Timeline
Ln 1, Col 1 Spaces: 4 UTF-8 LF Python Python 3.14.6 Go Live


ST
SecureTunnel
NETWORK SECURITY LAB
EDUCATIONAL SIMULATOR
PROTOCOL EXPLORER / 01
See how secure
connections are built.
Step through the handshake. Follow the messages. Understand what happens before private data travels across a network.
TLS / SSL
SSH
IPsec / ESP
TLS / SSL
Key exchange & handshake
SSH
Remote access handshake
IPsec / ESP
Packet protection modes
INTERACTIVE SIMULATION
TLS handshake / 00 of 05
Restart


CLIENT
Browser
Your device
Not secured
HANDSHAKE CHANNEL
Messages appear here as you advance
SERVER
Web Server
Destination host
Not secured
01 Client Hello
02 Server Hello & Certificate
03 Client Key Exchange
04 Session Key Generation
05 Finished
STEP EXPLANATION
Ready to establish a connection
Start the simulation, then advance one step at a time to inspect the protocol messages and cryptographic operations.
Start simulation
5 steps remaining
HANDSHAKE LOG
0
Waiting for handshake events.
AWAITING NEXT EVENT
TEST THE SECURE CHANNEL
After the handshake, send a sample message through AES-GCM authenticated encryption.
Hello over SecureTunnel!
Encrypt & send

Algortihms

Project 1 : "CryptoVault" - A Public-Key Cryptography Digital Signature Toolkit

Algorithm 1: RSA (Rivest–Shamir–Adleman) Purpose: Public-key encryption and digital signatures. 1. Select two large prime numbers, p and q. 2. Calculate $n = pq$ and $\phi(n) = (p-1)(q-1)$. 3. Choose a public exponent e that is relatively prime to $\phi(n)$. 4. Calculate the private exponent d, satisfying $ed \equiv 1 \pmod{\phi(n)}$. 5. Encrypt a message representative m using $c = m^e \bmod n$. 6. Decrypt using $m = c^d \bmod n$. 7. For signatures, sign a message digest with the private key and verify it with the public key.

Algorithm 2: Diffie–Hellman Key Exchange Purpose: Establishes a shared secret over a public communication channel. 1. Agree on public parameters: a large prime p and generator g. 2. Alice selects a private value a and computes $A = g^a \bmod p$. 3. Bob selects a private value b and computes $B = g^b \bmod p$. 4. Alice and Bob exchange their public values A and B. 5. Alice computes $S = B^a \bmod p$. 6. Bob computes $S = A^b \bmod p$. 7. Both obtain the same shared secret, which can be processed through a key-derivation function to produce a symmetric session key.

Algorithm 3: ElGamal Digital Signature Purpose: Provides message authentication and integrity verification. 1. Select a suitable prime p and generator g. 2. Choose a private key x and compute the public key $y = g^x \bmod p$. 3. Compute the SHA-512 digest of the message. 4. Select a fresh, secret signing value k that is relatively prime to $p-1$. 5. Calculate $r = g^k \bmod p$. 6. Calculate $s = k^{-1}(H(m) - xr) \bmod (p-1)$. 7. Verify the signature using the public key and the message digest.

Project 2: "SecureTunnel" - A Transport Layer Security (TLS/SSH) Visualizer

Algorithm 1: TLS Handshake Simulation Purpose: Establishes a secure channel between a client and server. 1. Client Hello: The client sends supported protocol versions, cryptographic capabilities, and other negotiation parameters. 2. Server Hello: The server selects compatible parameters and provides the information needed for key establishment. 3. Certificate Verification: The client checks the server's certificate, identity, validity, and trust chain. 4. Key Establishment: The client and server establish shared secret material using the negotiated key-exchange method. 5. Session-Key Derivation: Both parties derive symmetric traffic keys using a key-derivation function. 6. Finished Messages: Both sides verify handshake integrity and confirm that the negotiated handshake state matches. 7. Secure Communication: Application data is protected using symmetric authenticated encryption.

Algorithm 2: SSH Connection Establishment Purpose: Creates an authenticated, encrypted connection for remote access and secure data transfer. 1. Protocol Version Exchange: Client and server identify their SSH protocol versions. 2. Algorithm Negotiation: Both parties select compatible key-exchange, host-key, encryption, and

integrity algorithms. 3. Key Exchange: The parties use a method such as Diffie–Hellman or elliptic-curve Diffie–Hellman to establish shared secret material. 4. Server Authentication: The client verifies the server's host-key signature against a trusted or previously recorded host key. 5. Session-Key Derivation: Both sides derive keys for encrypting and authenticating traffic. 6. User Authentication: The client authenticates using a supported method, such as a password or public key. 7. Secure Channel Established: The connection can carry protected remote commands, data, or other supported services.

GitHub Url

`https://github.com/Sahasra-67/IAS_ALM2_Projects`